
Ollama vs vLLM in Production

- 1 Why your local LLM engine falls over under load
- 2 When to switch architectures for real-world traffic

The Concurrency Bottleneck

- 1 Ollama uses static slots and sequential FIFO queuing
- 2 At 50 concurrent users, p99 latency jumps from 3s to 25s

Continuous Batching Flow

- 1 vLLM dynamically allocates KV cache per iteration
- 2 GPU cores stay fully saturated instead of waiting on idle slots



Production Config Pattern

- 1 Cap sequence limits and use AWQ quantization to stabilize memory

```
main.py

from vllm import LLM

llm = LLM(
    model="meta-llama3-8B",
    quantization="AWQ",
    max_num_seqs=50
)
```

The Memory Trade-Off

- 1 vLLM: 6x throughput gain, consumes ~21GB VRAM at peak
- 2 Ollama: Flat ~5GB VRAM usage, but starves concurrent requests

The Engineering Rule

- 1 Ollama for laptops and internal single-user tools
- 2 vLLM for user-facing, high-concurrency production endpoints

